Bölüm 5 Yöntemler

Açılış Problemi

1 ile 10 arasındaki, 20 ile 30 arasındaki ve ___

35 ile 45 arasındaki tam sayıların toplamını bulun.

Problem

```
int toplam = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
System.out.println("1'den 10'a kadar toplam " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
  sum += i;
System.out.println("20'den 30'a kadar toplam " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
  sum += i;
System.out.println("35'ten 45'e kadar toplam " + sum);
```

Problem

```
int sum = 0;
for (int i = 1; i <= 10; i++)
  sum += i;
System.out.println("1'den 10'a kadar toplam " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
  sum += i;
System.out.println("20'den 30'a kadar toplam " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
  sum += i;
System.out.println("35'ten 45'e kadar toplam " + sum);
```

Çözüm

```
public static int sum(int i1, int i2) {
  int sum = 0;
  for (int i = i1; i <= i2; i++)
    sum += i;
  return sum;
}

public static void main(String[] args) {
  System.out.println("1'den 10'a kadar toplam " + sum(1, 10));
  System.out.println("20'den 30'a kadar toplam " + sum(20, 30));
  System.out.println("35'ten 45'e kadar toplam " + sum(35, 45));
}
```

Metot Tanımlama

Bir metot, bir işlemi gerçekleştirmek için bi
r araya getirilmiş ifadeler topluluğudur.

Bir metot tanımlayın

modifier

return value type

method name

formal parameters

metot başlık

```
public static int max(int num1, int num2) {
```

metot beden

```
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

parametre listesi

metot imzası

dönüş değeri

Bir metodu çağırın

```
int z = max(x, y);
```

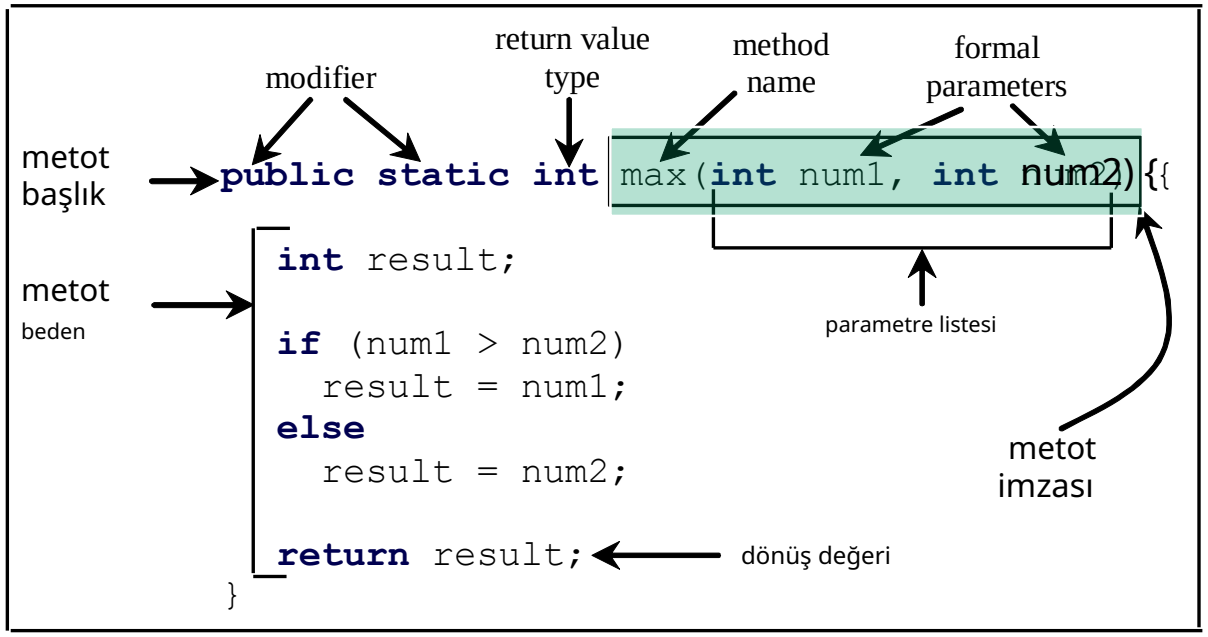
gerçek parametreler (argümanlar)

Metot İmzası

Method signature, metod adının ve

parametre listesinin kombinasyonudur.

Bir metot tanımlayın



Bir metodu çağırın

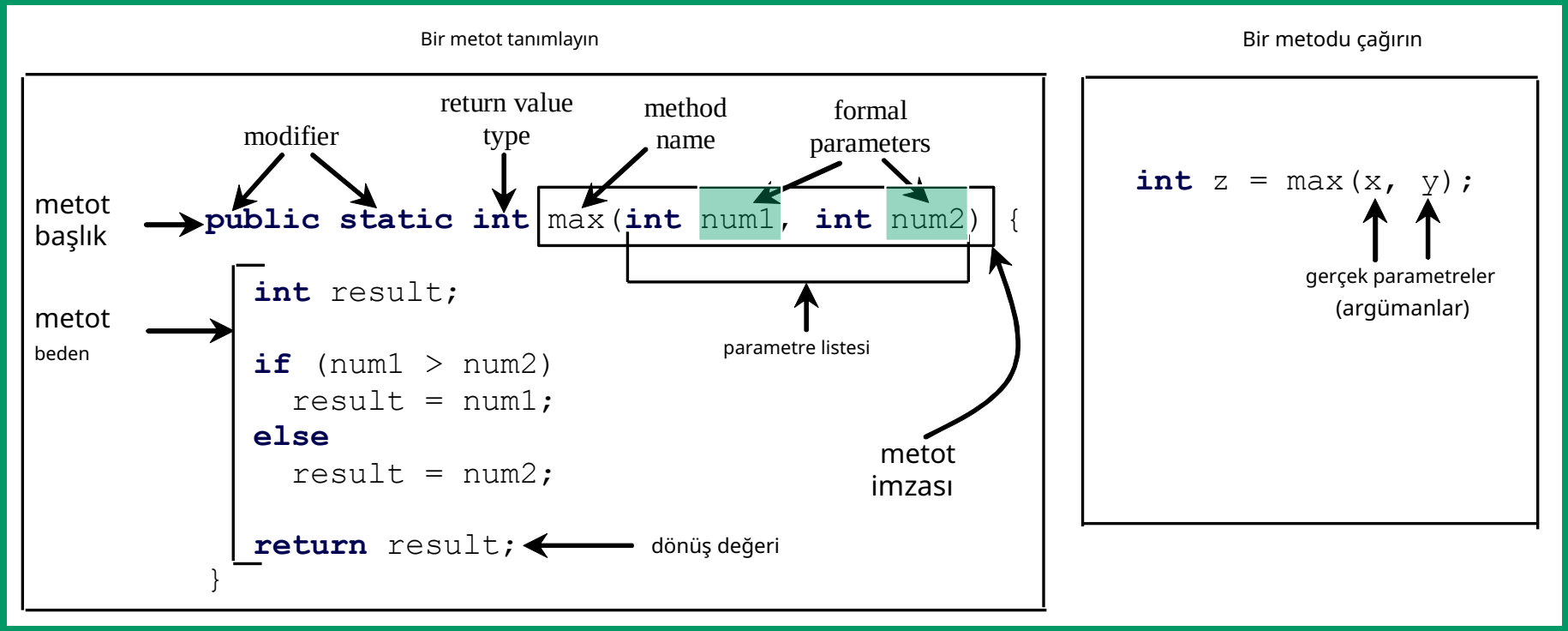
```
int z = max(x, y);
```

gerçek parametreler (argümanlar)

Formal Parameters

Metod başlığında tanımlanan değişkenler

formal parameters olarak bilinir.



Gerçek Parametreler

Bir yöntem çağrıldığında, bir değeri parametreye geçirirsiniz. Bu değer *gerçek parametre veya argüman* olarak adlandırılır.

Bir metot tanımlayın

modifier

return value type

method name

formal parameters

metot başlık

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

metot beden

parametre listesi

metot imzası

dönüş değeri

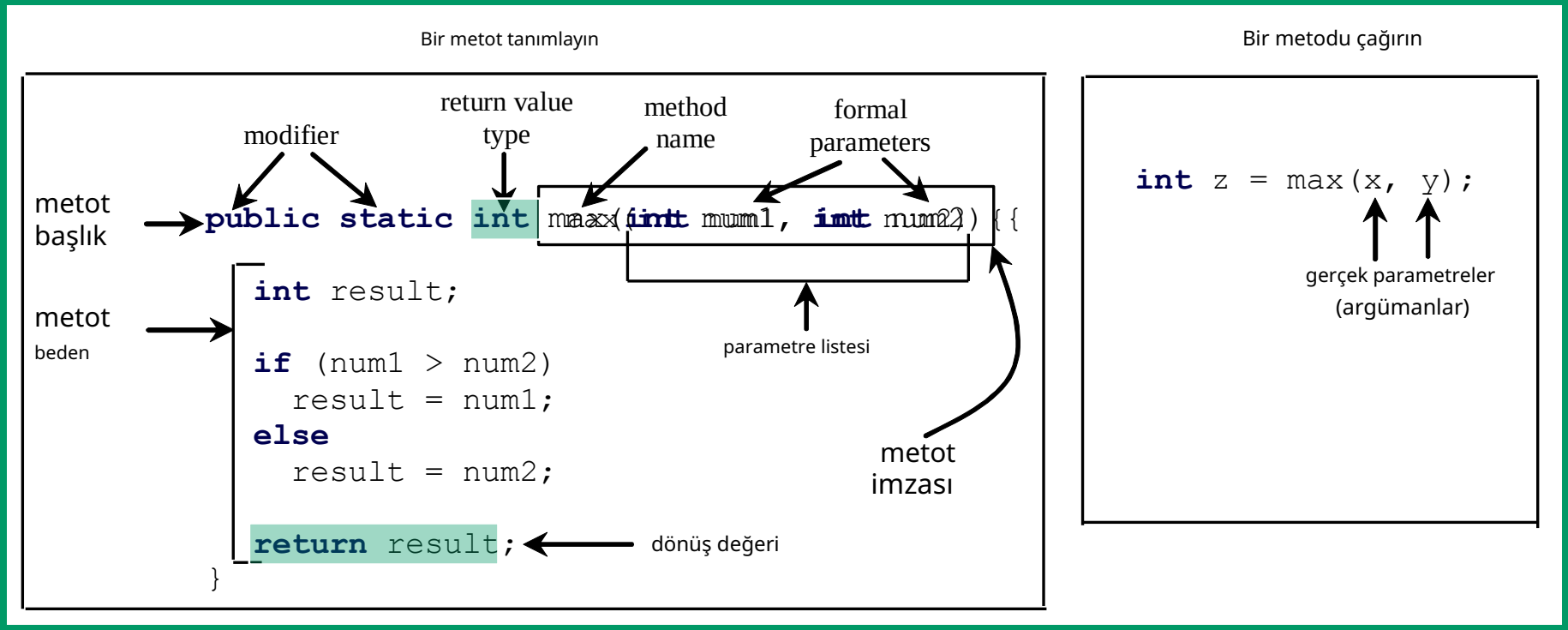
Bir metodu çağırın

```
int z = max(x, y);
```

gerçek parametreler (argümanlar)

Return Value Type

Bir yöntem bir değer döndürebilir. returnValueType, veri türüdür. metodun döndürdüğü değerin. Eğer metod bir değer, returnValueType anahtar kelimesi void'tır. Örneğin, main metodundaki returnValueType void'tur.



Çağrı Yöntemleri

`max` yöntemini test etme

Bu program, bir max yönteminin çağrılmasını göstermektedir.

en büyük `int` değerini döndürmek

TestMax

Koş

animasyon

Yöntemleri Arama, devam.

i değerini geç

j değerini geç

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

i şimdi 5

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

j şimdi 2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

max(i, j) çağrısı yap

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

max(i, j) çağrısı yap
i değerini num1'e geçir
j değerini num2'ye geçir

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

değişken result'u tanımla

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

(num1 > num2) doğru çünkü num1
5 ve num2 2'dir

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

result şimdi 5

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

result'ı döndür, bu da 5'tir

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
pu      tatic int max(int num1, int num2) {
        esult;

   num1 > num2)
   esult = num1;
  se
   result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

max(i, j)'yi döndür ve ata
k'ya dönüş değeri

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

animasyon

İzleme Yöntemi Çağrısı

Yazdırma ifadesini çalıştır

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

DİKKAT

Bir değer döndüren yöntem için bir return ifadesi gereklidir. The aşağıda (a) gösterilen yöntem mantıksal olarak doğrudur, ancak bir derleme hatası çünkü Java derleyicisi bunun mümkün olduğunu düşünüyor bu yöntem herhangi bir değer döndürmez.

```
public static int işaret(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else if (n < 0)
    return -1;
}
```

(a)

Olmalıdır →

```
public static int işaret(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else
    return -1;
}
```

(b)

Bu sorunu çözmek için, derleyicinin çalışabilmesi için (a) içindeki *if (n < 0)* ifadesini silin.

if'in nasıl olduğuna bakılmaksızın bir return ifadesinin ulaşılması gerektiğini görün

ifade değerlendirilir.

Diğer Sınıflardan Yöntemleri Yeniden Kullanma

NOT: Yöntemlerin faydalarından biri yeniden kullanımdır. max yöntemi TestMax dışında herhangi bir sınıftan çağrılabilir. Eğer yeni bir Test sınıfı oluşturursanız, max yöntemini ClassName.methodName (örneğin, TestMax.max) kullanarak çağırabilirsiniz.

void Yöntem Örneği

Bu tür bir yöntem değer döndürmez. Yöntem bir takım işlemler gerçekleştirir.

TestVoidMethod

Koş

Parametre Geçişi

```
public static void nPrintln(String message, int n) {
  for (int i = 0; i < n; i++)
    System.out.println(message);
}
```

Farz edelim ki, metodu şu şekilde çağırıyorsunuz:

nPrintln(“Java’ya Hoş Geldiniz”, 5);

Çıktı nedir?

Farz edelim ki, metodu şu şekilde çağırıyorsunuz:

nPrintln(“Bilgisayar Bilimleri”, 15);

Çıktı nedir?

Değerle Geçiş

Bu program, değerlerin

yöntemlere geçirilmesini göstermektedir.

Artış

Koş

Değerle Geçiş

Değer ile Test Geçişi

Bu program, değerlere

yöntemlere geçişi göstermektedir.

DeğerİleGeçişTesti

Koş

Yükleme Yöntemleri

`max` Yöntemini Yükleme

```
public static double max(double num1, double
  num2) {
   if (num1 > num2)
     return num1;
   else
     return num2;
}
```

TestMethodOverloading

Koş

Ambiguous Invocation

Bazen bir yöntemin çağrısı için iki vey
a daha fazla uygun eşleşme olabilir, anc
ak derleyici en özel eşleşmeyi belirleyemez.
Bu, *belirsiz çağrı* olarak adlandırılır.
Belirsiz
çağrı bir derleme hatasıdır.

Ambiguous Invocation

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));
  }

  public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }

  public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

Yerel Değişkenlerin Kapsamı

Yerel bir değişken: bir

metodun içinde tanımlanan bir değişken.

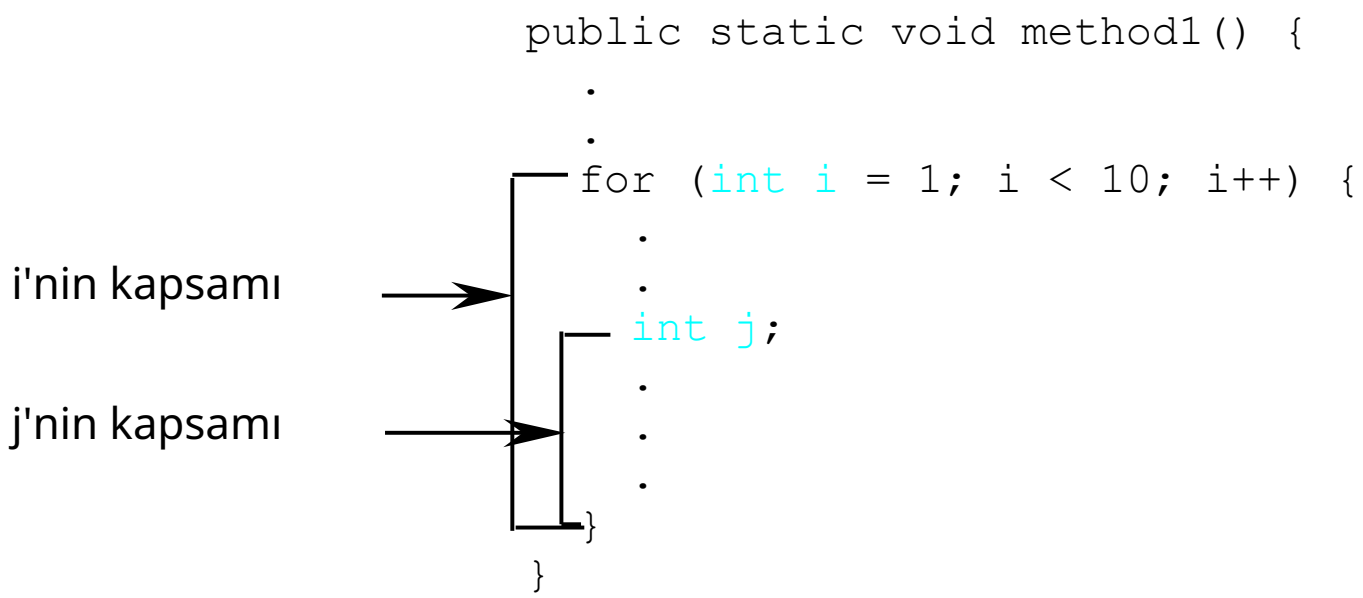
Kapsam: değişkenin

başvurulabileceği programın kısmı.

Yerel bir değişkenin kapsamı, değişkenin tanımından başlar ve içeren blok bitene kadar devam eder. Yerel değişken kullanılmadan önce tanımlanmal ıdır.

Yerel Değişkenlerin Kapsamı, devam.

Bir for döngüsünün başlangıç eylem kısmında tanımlanan bir <u>değ</u>işken header, döngünün tamamında kapsamı vardır. Ancak bir değişken bir döngü gövd<u>esi</u>nde tanımlanan, kapsamı sınırlıdır değişkenin tanımından döngü gövdesine ve bloğun sonuna kadar değişkeni içeren.



Yerel Değişkenlerin Kapsamı, devam.

İki

iç içe olmayan blokta i'yi tanımlamak sorun değil

```
public static void method1() {
  int x = 1;
  int y = 1;

  for (int i = 1; i < 10; i++) {
    x += i;
  }

  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

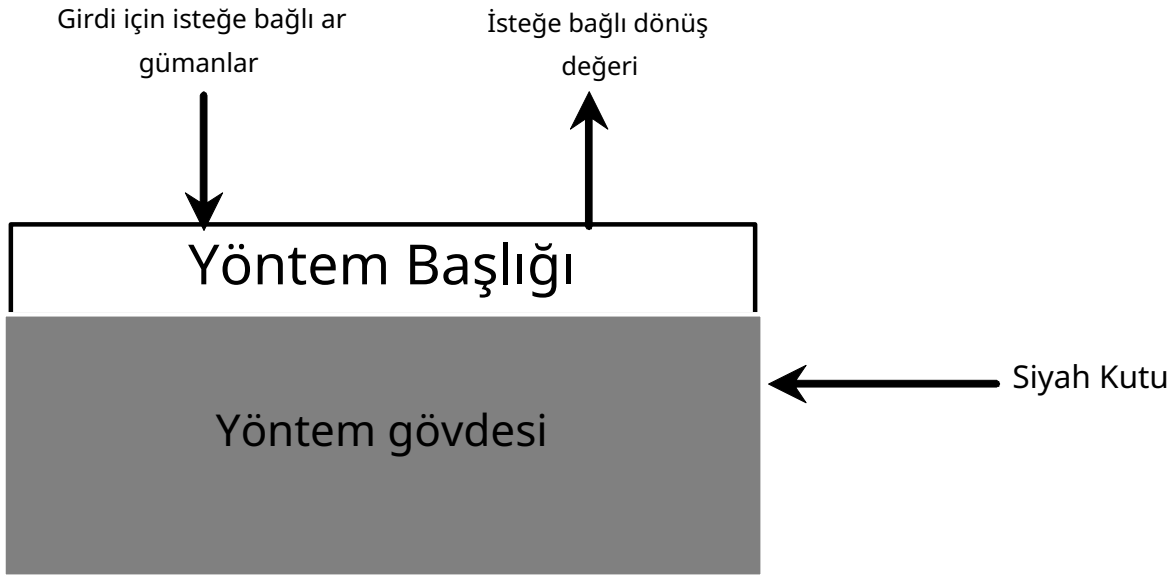
İki iç içe blokta i'yi tanımlamak yanlış tır

```
public static void method2() {

  int i = 1;
  int toplam = 0;

    for (int i = 1; i < 10; i+
      toplam += i;
  }
}
```

Metot Soyutlama

Yöntem gövdesini bir kara kutu olarak düşünebilirsiniz

yöntemin detaylı uygulamasını içeren



Yöntemlerin Faydaları

- Bir yöntemi bir kez yazın ve her yerde yeniden kullanın.
- Bilgi gizleme. Kullanıcıdan uygulamayı gizle.
- Kompleksliği azalt.

Matematik Sınıfı

- Sınıf sabitleri:
  - `PI`
  - `E`
- Sınıf yöntemleri:
  - Trigonometri Yöntemleri
  - Üs Yöntemleri
  - Yuvarlama Yöntemleri
  - min, max, abs ve rastgele Yöntemler

Trigonometrik Yöntemler

- `sin(double a)`
- `cos(double a)`
- `tan(double a)`
- `acos(double a)`
- `asin(double a)`
- `atan(double a)`

Radians

toRadians(90)

Örnekler:

Math.sin(0) 0.0 döner

`Math.sin(Math.PI / 6)`
0.5 döner

`Math.sin(Math.PI / 2)`
1.0 döner

Math.cos(0) 1.0 döner

`Math.cos(Math.PI / 6)`
0.866 döner

`Math.cos(Math.PI / 2)`
0 döner

Üslü Yöntemler

- `exp(double a)`
  `e` sayısının `a` üssünü döndürür.
- `log(double a)`
  a sayısının doğal logaritmasını döndürür.
- `log10(double a)`
  a sayısının 10 tabanındaki logaritmasını döndür ür.
- `pow(double a, double b)`
  a sayısını b üssüne yükseltir.
- `sqrt(double a)`
  √`a` döner.

Örnekler:

Math.exp(1) 2.71 döner
Math.log(2.71) 1.0 döner
Math.pow(2, 3) 8.0 döner
Math.pow(3, 2) 9.0 döner
Math.pow(3.5, 2.5) döner
`22.91765`
Math.sqrt(4) 2.0 döner
Math.sqrt(10.5) 3.24 döner

Yuvarlama Yöntemleri

- `double ceil(double x)`
  x en yakın tam sayıya yukarı yuvarlanır. Bu tam sayı bir double

  değer olarak döner.

- `double floor(double x)`
  x, en yakın tam sayıya aşağı yuvarlanır. Bu tam sayı,

  double değeri olarak döndürülür.

- `int round(float x)`
- `long round(double x)`

Yuvarlama Yöntemleri Örnekleri

Math.ceil(2.1) 3.0 döndürür
Math.ceil(2.0) 2.0 döndürür
Math.ceil(-2.0) -2.0 döndürür
Math.ceil(-2.1) -2.0 döndürür
Math.floor(2.1) 2.0 döndürür
Math.floor(2.0) 2.0 döner
Math.floor(-2.0) -2.0 döner
Math.floor(-2.1) -3.0 döner
Math.round(2.6f) 3 döner
Math.round(2.0) 2 döner
Math.round(-2.0f) -2 döner
Math.round(-2.6) -3 döner

min, max ve abs

- `max(a, b)` ve `min(a, b)`

  İki parametrenin maksimumunu veya minimumunu döner.

- `abs(a)`

  Returns the absolute value of the parametre.

- `random()`

  Rastgele `double` değeri döndürür [0.0, 1.0) aralığında.

Örnekler:

Math.max(2, 3) 3 döner

Math.max(2.5, 3) döner
`3.0`

`Math.min(2.5, 3.6)`
2.5 döner

Math.abs(-2) 2 döner

Math.abs(-2.1) döner
`2.1`

Rastgele Yöntem

0.0 veya daha büyük ve daha az rastgele bir double değeri üretir.

1.0'dan küçük (0 <= Math.random() < 1.0).

Örnekler:

```
(int)(Math.random() * 10)
```
→ 0 ile 9 arasında rastgele bir tam sayı döndürür.

```
50 + (int)(Math.random() * 50)
```
→ 50 ile 99 arasında rastgele bir tam sayı döndürür.

Genel olarak,

```
a + Math.random() * b
```
→ a ile a + b arasında, a + b hariç, rastgele bir sayı döndürür.